

TP2 — Informe

Sistema de Detección y Clasificación de Razas de Perros

IA 5.2 — Computer Vision · Tecnicatura Universitaria en Inteligencia Artificial (TUIA), UNR

Alumno 1: Maximiliano Romano

Alumno 2: Facundo Molina

1. Explicación completa del pipeline

Etapa 1 - Embeddings y búsqueda por similitud

Cada imagen se lee con OpenCV y se pasa por una ResNet50 preentrenada a la que se le saca la capa final de clasificación, utilizándola como extractor de características. El resultado es un embedding de 2048 dimensiones que resume la forma, textura y rasgos visuales del perro.

Estos vectores se guardan en PostgreSQL con la extensión pgvector, procesando las imágenes en lotes de 32.

Ante una nueva consulta, el sistema calcula el embedding de la imagen y busca sus vecinos más cercanos combinando similitud coseno y distancia euclidiana (L2). La predicción final se resuelve por voto mayoritario ponderado por cercanía, y si el vecino más próximo está demasiado lejos se activa un umbral que devuelve desconocido en lugar de forzar una clasificación incorrecta. La calidad del ranking se valida con NDCG@10.

Etapa 2 - Clasificación supervisada

Se entrenaron y compararon dos clasificadores supervisados para las 70 razas un ResNet18 con fine-tuning y una CNN diseñada desde cero (CNN Custom).

Etapa 3 - Pipeline completo de detección y clasificación

Esta etapa integra parte de la Etapa 2 para resolver el caso de uso real.

Detección: YOLOv8 localiza objetos de la clase "perro" en la imagen completa y genera un bounding box por cada instancia con buena confianza.

Los bounding box se recortan para no exceder los márgenes de la imagen.

Preprocesamiento del recorte: se convierte a RGB, se redimensiona a 224×224 px y se normaliza.

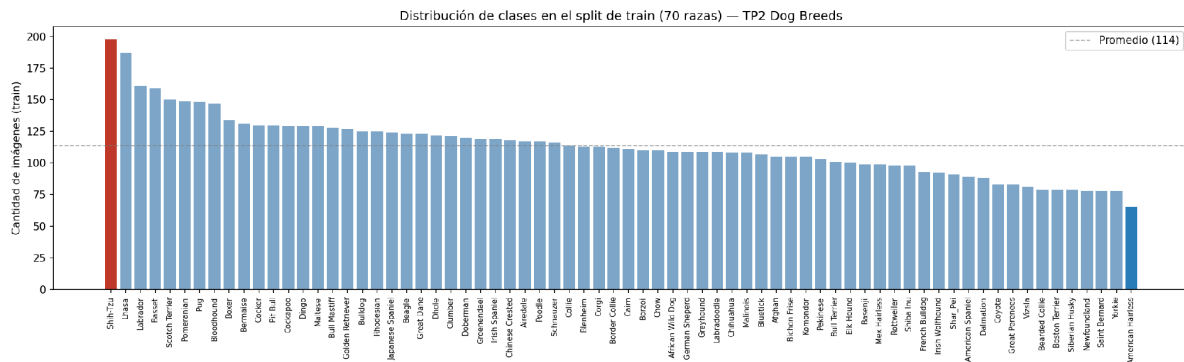
Clasificación: el recorte se pasa por el ResNet18 fine-tuned, que devuelve la raza más probable entre las 70 clases junto con su confianza.

2. Dataset

El dataset contiene 70 razas de perros (Kaggle, "70 Dog Breeds Image Data Set"), descargado con scripts/download_dataset.py y organizado en tres splits: train, valid y test.

El split de entrenamiento tiene 7946 imágenes.

El dataset presenta un desbalance leve en el split de entrenamiento, la clase con más imágenes es Shih-Tzu con 198 y la clase con menos imágenes es American Hairles con 65.



3. Preprocesamiento

Eta 1 (embeddings)

Redimensión (Resize((256, 256))): Estandariza las imágenes de entrada de diversas resoluciones a un tamaño homogéneo antes del recorte, optimizando el uso de memoria en la GPU.

Recorte Central (CenterCrop(image_size)): Extrae la región central de tamaño fijo (ej. 224 \224). Esto remueve artefactos de los bordes y asegura la compatibilidad con las capas totalmente conectadas del backbone.

Normalización: Aplica los parámetros estadísticos de ImageNet: mean=[0.485, 0.456, 0.406] y std=[0.229, 0.224, 0.225]. Esto centra la distribución de los gradientes acelerando la convergencia del optimizador.

Conversión de Espacio de Color: Ajusta la lectura nativa de OpenCV (BGR) a formato RGB mediante cv2.cvtColor antes de instanciar objetos PIL, evitando inversiones en los canales de color que arruinarían las inferencias del modelo predictivo.

SIMILARITY_THRESHOLD=0.55

Etapas 2 y 3 (clasificación y detección)

El recorte de cada perro se convierte a RGB, se redimensiona a 224×224 píxeles que es el tamaño de entrada de ResNet18 y se normaliza por canal de color.

Las tres etapas usan la media y desvío estándar de ImageNet, mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225].

conf_threshold=0.25

Data augmentation

Se aplicó data augmentation en la etapa 2, recorte aleatorio (RandomResizedCrop) y flip horizontal (RandomHorizontalFlip). Se agrego rotación leve (+-15°) y variación de brillo/contraste/saturación (ColorJitter).

4. Justificación de los modelos elegidos

Modelo de embeddings (Etapa 1) - ResNet50 preentrenada

Se eligió ResNet50 preentrenada en un dataset masivo de imágenes generales, usándola únicamente como extractor de características, sin su capa de clasificación. Esto permite obtener una representación visual del perro sin necesidad de entrenar nada en esta etapa donde no se requiere una etiqueta de clase exacta sino una noción.

Clasificadores (Etapa 2) - ResNet18 fine-tuned vs. CNN Custom

ResNet18: se parte de los pesos preentrenados en ImageNet, que ya capturan bordes, formas y texturas generales, y se reemplaza la capa final para adaptarla a las 70 razas. Se optimizan todos los parámetros de la red durante el entrenamiento, con Adam y learning rate bajo (0.001) lo que permite ajustar también las capas convolucionales preentrenadas a las imagenes específicas de razas de perros.

CNN Custom: arquitectura de 5 bloques convolucionales con filtros 3×3, batch normalization, activación y max-pooling, con canales de 32, 64, 128, 256, 256 y Dropout del 30% para reducir el sobreajuste.

Detector (Etapa 3) - YOLOv8

Se eligió YOLOv8 por su capacidad frente a múltiples instancias en una misma imagen y ya que fue entrenado sobre datasets generales grandes que incluyen la categoría perro.

Análisis de la Arquitectura:

- ResNet18: Al entrenar solo la última capa, el proceso es rápido, gasta muy poca memoria y llega a resultados muy buenos en pocas épocas. Es la mejor opción para la aplicación real.
- CNN Custom: Requiere demasiado tiempo y esfuerzo de entrenamiento para alcanzar niveles aceptables de precisión
- YOLO: Se eligió la versión YOLOv8 porque es rápida y consume muy pocos recursos.

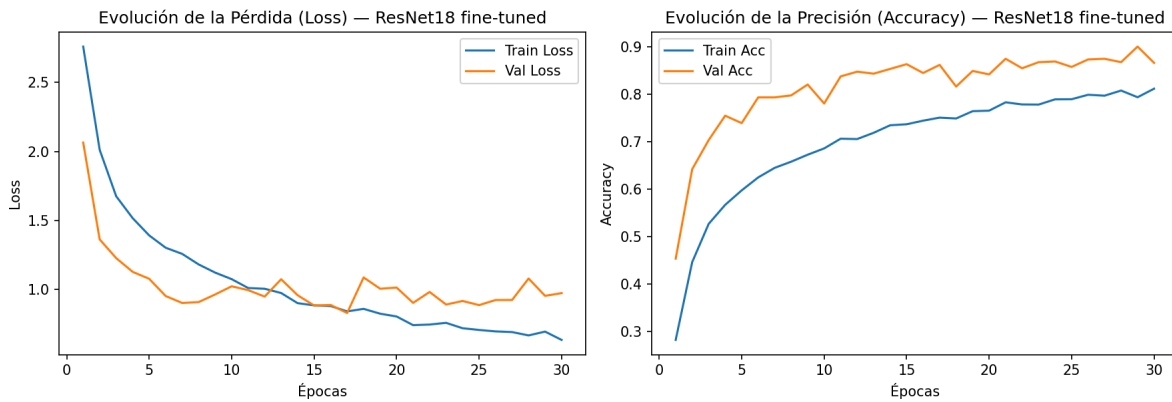
5. Proceso de entrenamiento e hiperparámetros

Ambos modelos se entrenaron con el mismo optimizador y la misma cantidad de épocas.

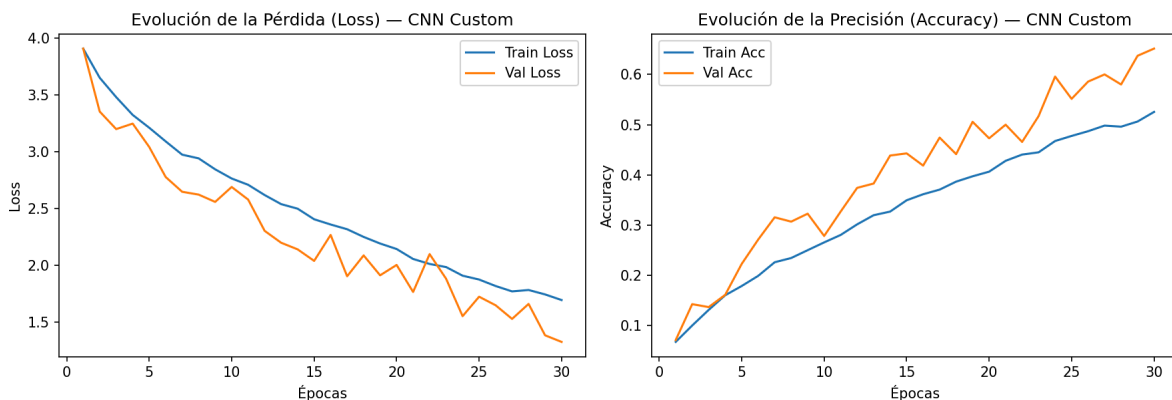
Modelo	Optimizador	Learning rate	Scheduler	Batch size	Épocas
ResNet18 fine-tuned	Adam	0.001	No	32	30
CNN Custom	Adam	0.001	No	32	30

Curvas de entrenamiento

ResNet18: convergencia estable. La accuracy de entrenamiento pasa de 28% (época 1) a 81% (época 30); la de validación pasa de 45% a un máximo de 90% (época 29), cerrando en 86,6%. La curva de validación se mantiene por encima de la de entrenamiento durante todo el proceso, sin señales de overfitting, el val loss oscila bastante entre épocas.



CNN Custom: El accuracy de entrenamiento llega a 52,5% y la de validación a 65,1% en la época 30, ambas curvas todavía van en ascenso al final, lo que sugiere que el modelo no llegó a converger del todo en 30 épocas, pero por una limitación de tiempo y recursos se limitó a 30 épocas.



6. Resultados obtenidos

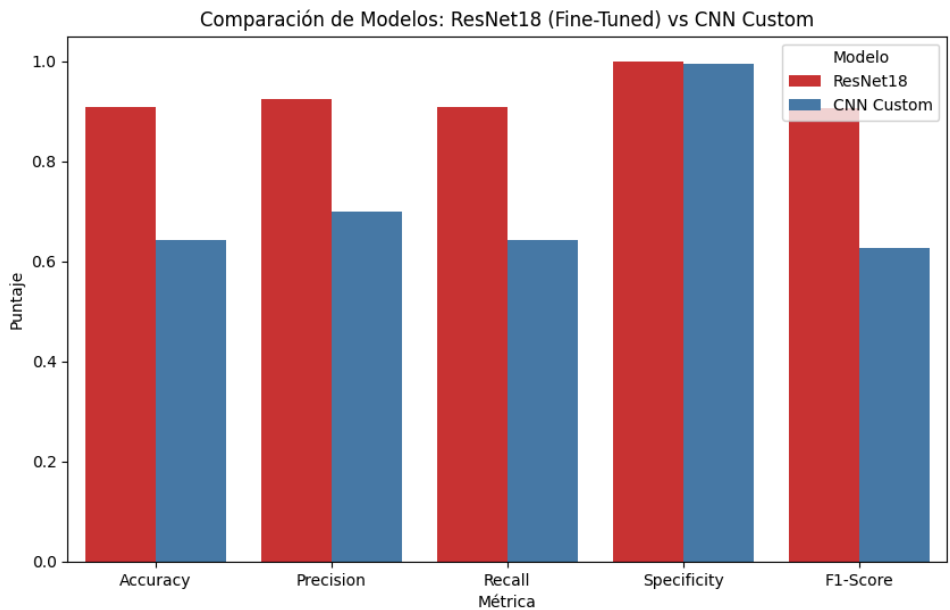
Etapa 1 — NDCG@10

La métrica NDCG@10 evalúa los primeros 10 resultados devueltos por la búsqueda, premiando que los perros de la misma raza aparezcan en los primeros lugares del ranking y penalizando que aparezcan al final. El puntaje va de 0 a 1, donde 1 representa una búsqueda perfecta.

Etapa 2 — Métricas de clasificación

Métricas obtenidas con `evaluate_classifier()` sobre el split de test:

Modelo	Accuracy	Precision	Recall	Specificity	F1-Score
ResNet18	0.9086	0.9233	0.9086	0.9987	0.9056
CNN Custom	0.6429	0.6988	0.6429	0.9948	0.6263



7. Comparación entre enfoques

Búsqueda por similitud vs. clasificación supervisada

El primer enfoque consiste en una búsqueda por similitud, donde la imagen de consulta se transforma en un vector de características y se compara con los vectores almacenados en la

base de datos mediante búsquedas de vecinos más cercanos utilizando pgvector. En lugar de devolver una única etiqueta, este enfoque recupera las imágenes más similares según la distancia entre embeddings.

El segundo enfoque es una clasificación supervisada, en la que un modelo entrenado (ResNet18) recibe una imagen y predice directamente la raza más probable entre las clases conocidas. Este método dio una alta precisión cuando las imágenes pertenecen a las categorías utilizadas durante el entrenamiento.

La principal diferencia entre ambos métodos es que la clasificación supervisada busca identificar directamente la clase de la imagen, mientras que la búsqueda por similitud encuentra ejemplos visualmente parecidos.

ResNet18 fine-tuned vs. CNN Custom

ResNet18: 90,9% de accuracy contra 64,3% de CNN Custom,

Por costo/beneficio, ResNet18 resulta la opción más correcta para este caso, entrena en pocas épocas, consume poca memoria y logra buen desempeño general. La CNN Custom requiere muchas más épocas para tener resultados aceptables y con 30 épocas no alcanzo un buen resultado.

8. Problemas encontrados y soluciones implementadas

- Pruebas para mejora de métricas: Para el caso de la ResNet18 finetuned y CNN-Custom, tal como se observa en las fotos que muestran las métricas obtenidas en esas pruebas realizadas, intentamos ir modificando parámetros como learning rates, dropouts, etc y no logramos mejorar las mismas. Inclusive en esta última agregamos un Early Stopping que no fue gran ayuda para reducir tiempos.

```
from lib.bootstrap import build_classifier

classifier = build_classifier(settings)
classifier.set_active_model("resnet18_finetuned")
classifier.train_classifier()
```

```
*** Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root/.cache/torch/hub/checkpoints/resnet18-f37072fd.pth
100%|██████████| 44.7M/44.7M [00:00<00:00, 198MB/s]
Epoch 1/30 | Train Loss: 4.2977 Acc: 0.0320 | Val Loss: 3.9268 Acc: 0.0529
Epoch 2/30 | Train Loss: 3.9288 Acc: 0.0462 | Val Loss: 4.0437 Acc: 0.0343
Epoch 3/30 | Train Loss: 3.8507 Acc: 0.0614 | Val Loss: 3.7879 Acc: 0.0686
Epoch 4/30 | Train Loss: 3.7297 Acc: 0.0861 | Val Loss: 4.1997 Acc: 0.0686
Epoch 5/30 | Train Loss: 3.5935 Acc: 0.1077 | Val Loss: 3.3495 Acc: 0.1400
Epoch 6/30 | Train Loss: 3.5106 Acc: 0.1241 | Val Loss: 3.4101 Acc: 0.1471
Epoch 7/30 | Train Loss: 3.4857 Acc: 0.1253 | Val Loss: 3.4268 Acc: 0.1200

KeyboardInterrupt                                Traceback (most recent call last)
/tmp/ipykernel_1623/670841423.py in <cell line: 0>()
      3 classifier = build_classifier(settings)
      4 classifier.set_active_model("resnet18_finetuned")
----> 5 classifier.train_classifier()
```

```
Early Stopping: 3 de 5 épocas sin mejoras.
Epoch 21/30 | Train Loss: 2.4766 Acc: 0.3341 | Val Loss: 2.0529 Acc: 0.4500
Epoch 22/30 | Train Loss: 2.4445 Acc: 0.3393 | Val Loss: 1.9759 Acc: 0.4857
Epoch 23/30 | Train Loss: 2.3790 Acc: 0.3568 | Val Loss: 2.0702 Acc: 0.4443
Early Stopping: 1 de 5 épocas sin mejoras.
Epoch 24/30 | Train Loss: 2.3571 Acc: 0.3662 | Val Loss: 2.0032 Acc: 0.4643
Early Stopping: 2 de 5 épocas sin mejoras.
Epoch 25/30 | Train Loss: 2.3181 Acc: 0.3706 | Val Loss: 2.0835 Acc: 0.4300
Early Stopping: 3 de 5 épocas sin mejoras.
Epoch 26/30 | Train Loss: 2.2775 Acc: 0.3769 | Val Loss: 1.8969 Acc: 0.4743
Epoch 27/30 | Train Loss: 2.2533 Acc: 0.3803 | Val Loss: 1.9251 Acc: 0.4886
Early Stopping: 1 de 5 épocas sin mejoras.
Epoch 28/30 | Train Loss: 2.2730 Acc: 0.3782 | Val Loss: 1.7698 Acc: 0.5257
Epoch 29/30 | Train Loss: 2.1955 Acc: 0.3969 | Val Loss: 1.7006 Acc: 0.5600
Epoch 30/30 | Train Loss: 2.1643 Acc: 0.4081 | Val Loss: 2.0197 Acc: 0.4643
Early Stopping: 1 de 5 épocas sin mejoras.
Cargar los mejores pesos encontrados
Checkpoint guardado en: models/cnn_custom.pth
{'accuracy': 0.53,
 'precision': 0.5596936106058176,
 'recall': 0.5300000000000001,
 'specificity': 0.993188405795662,
 'f1': 0.5105530094454918,
 'confusion_matrix': [[8,
```

9. Modificaciones fuera de las funciones indicadas

Se mantuvo un respeto absoluto de la arquitectura base provista por la cátedra. No se modificó ninguna línea de código fuera de los alcances permitidos. Los problemas fueron resueltos en su totalidad de forma interna y encapsulada dentro del cuerpo de la función asignada a nosotros en cada ejercicio según correspondiera.